P L A N

# Product Manager Plan

Step 3 of 5 — CIRCLES Recommendation + Blueprint Plan Mode

Project: Guest Profiling System · Repo: lordwhitefire/guest-profiling-system

Phase: 3 of 5 — Product Manager Planning

Method: PM agent CIRCLES + Blueprint skill Plan mode (80-area checklist)

Build order confirmed (Victor, 2026-11-08): Functionality → Business logic → Data → UI

Prepared by: Z.ai Builder Agent · Date: 2026-11-10

## 1. Plan-First Note

Per the PM agent's standing rule ("Plan First, Always"), this section records what we were asked to produce and what we will produce, in writing, before the work. The owner said "go" to Step #3 — that is the agreement.

What we were asked to produce: a Product Manager recommendation, using the CIRCLES method, on how to build the Guest Profiling System. The recommendation consumes the Step #1 Discovery Report and the Step #2 Behavioral Specification (the five spec cards) as its discovery corpus, and respects the project's hard constraints (read-only source, one-way sync, single-developer maintainable, NDPR / GDPR / CCPA compliance, no AI features, no automatic duplicate detection).

What we will produce: this document. A CIRCLES-shaped recommendation followed by a Blueprint Plan-mode walkthrough that marks every one of the 80 areas as Decided, Skip, or Pending. The document ends with a phased build plan that becomes the input to Step #4 (OpenSpec design + actual code).

PM agent boundary: this document recommends, it does not decide. Where the recommendation depends on owner input, the area is marked Pending with the specific question to resolve. The PM agent does not act on the product — no code, no config, no ticket. The build happens in Step #4, against the OpenSpec produced from this plan.

## 2. Executive Summary

Recommendation in one line: build the Guest Profiling System in four phases, each delivering independent value, each mapping to one or more of the five reverse-engineered behavioral spec cards. The single metric that defines success is the p95 receptionist round-trip — type a few characters, see matching profiles, click one, fill the registration form — under 500 milliseconds.

The four phases are: Phase 0 (foundation — schema, design system, project setup, decided from Cards 2 and 3), Phase 1 (read-side sync adapter per Card 1 + guest profile API per Card 2 + receptionist search screen), Phase 2 (browser-extension form-fill adapter per Card 5), and Phase 3 (additional PMS adapters — API-based like Mews or Cloudbeds, and Windows-desktop via a companion app). Phase 0 work is already decided by the spec cards; Phases 1–3 need OpenSpec design (Step #4) before any code lands.

Three decide-first areas from the Blueprint skill are settled in this document: constraints (area 75), multi-tenancy (area 49), and delete policy (area 59). The decisions are: single-tenant deployment in v1 but tenant_id column on every table from day one; soft delete with deleted_at timestamps for guest profiles, never hard delete. Section 8 has the full Plan-mode walkthrough.

What this document does NOT do: pick the hosting provider (Pending — owner's call), name the first hotel customer (Pending — owner's call), decide whether to open-source the resulting code (Pending — owner's call). Those are flagged in the 80-area checklist as Pending with the specific question to resolve.

## 3. CIRCLES — Comprehend

What is actually being asked, before any solving starts. The PM agent's comprehend step is deliberately conservative: do not jump to features, do not jump to solutions, establish the situation first.

### 3.1 The product we are planning

The Guest Profiling System is a hotel add-on that reads existing guest records from a hotel's Property Management System (PMS), presents live-filtered matching profiles as a receptionist types, and auto-fills the hotel's existing registration form on click. It works alongside the PMS without replacing it. It never writes to the hotel's database. It is custom-built per hotel, maintained by a single developer, with a shared core across hotels and a per-hotel adapter.

### 3.2 The situation we are in

Step #1 identified five open-source reference repositories whose runtime behavior is relevant to this project. Step #2 reverse-engineered each into a stack-agnostic behavioral spec card. We now have a complete behavioral specification for every runtime surface the system must reproduce — read-side sync, guest API contract, guest lifecycle state, messy source shape, and smart form-fill. We have not yet written any code, made any stack decisions binding, or produced any OpenSpec. This is the moment to plan.

### 3.3 The constraints we are operating under

Table 1: Project constraints carried into CIRCLES

| Constraint | Source | Effect on planning |
|---|---|---|
| Read-only access to hotel's PMS data | Project plan | Adapter is one-way sync (Card 1 contract); no write-back path |
| One-way data flow: source DB → our DB | User confirmation | Adapter is ETL-shaped, not query-translation |
| Single-developer maintainable | Project plan | Stack must be familiar to one person; documented; tests required |
| NDPR (Nigeria) + GDPR (EU) + CCPA (US) | Project context (Lagos TZ) | Privacy-by-design; data subject rights; documented retention |
| No AI features | Project plan | No LLM dependency; deterministic ranking only |
| No automatic duplicate detection | Project plan | Each source row maps to exactly one profile; no fuzzy merge |
| No spell correction | Project plan | Search uses prefix and substring match only |
| Frontend: art-directed, polished, modern | User design brief | Custom design system; no template look |
| Backend: from-scratch re-implementation | Step #2 license review | No source code imported from any of the 5 reference repos |

### 3.4 What is NOT being asked

Equally important to state. This plan does not cover: replacing the hotel's PMS, building a new check-in or check-out system, building a second registration process, building a standalone CRM, building AI features, building automatic duplicate detection, building spell correction, writing anything into the hotel's database. All explicitly out of scope per the project plan and reaffirmed here.

## 4. CIRCLES — Identify

Who the work is for, and for which side. The PM agent's identify step names every person the product touches, then says which side we are on.

### 4.1 The customer sides

Table 2: Customer sides and what each one wants

| Side | Who | What they want (stated) | What they need (underlying) |
|---|---|---|---|
| Primary user | Hotel receptionist | Auto-fill the registration form when a returning guest arrives | Reduce typing at check-in; look competent in front of the guest; not have to learn a new system |
| Buyer | Hotel owner / manager | Faster check-in; modernize the front desk | Better guest experience + lower staff workload + competitive parity with bigger chains, without PMS replacement cost |
| Data subject | Hotel guest | (does not interact with the system directly) | Their personal data is stored securely, used only for the purpose they gave it, and deletable on request |
| Gatekeeper | Hotel's PMS vendor | (not the customer; their permission is required for adapter read access) | Their PMS is not destabilized; no third-party writes; no reverse engineering of their code |
| Maintainer | Single developer (the owner of this project) | A maintainable codebase with clear adapter boundaries | Few moving parts; clear contracts; documented; testable in isolation |
| Regulator | NDPR / GDPR / CCPA authorities | (does not interact directly) | Lawful processing; documented consent; data subject rights honored; breach notification possible |

## 4.2 Which side we are on

We are on the receptionist's side first, and the buyer's side second. The receptionist's experience is the primary value — if the system makes their job harder, the buyer will not renew. The data subject's rights are a hard constraint, not a side. The maintainer's burden is a hard constraint too — the project explicitly says single-developer maintainable. The gatekeeper (PMS vendor) is someone we need permission from, not someone we serve.

## 4.3 Stakeholder map

Beyond the customer sides, the following stakeholders are involved in or affected by the build, and must be considered in the Plan-mode walkthrough:

- Hotel staff (housekeeping, billing, night audit) — indirect beneficiaries of faster check-in; not direct users of the system.
- Hotel IT (if any) — responsible for granting read-only DB credentials or API access; signs off on adapter install.

- PMS vendor — gatekeeper; may publish a public API (best case), grant read-only DB login (acceptable), or refuse access (we walk away).
- Open-source community — if the project goes open source (Pending decision), they contribute adapters for other PMS systems.

## 5. CIRCLES — Report

Name the customer's underlying need rather than their feature want. The PM agent's report step is the bridge from stated wants to the need that, if met, makes the customer's actual job better.

### 5.1 Receptionist's underlying need

Stated want: "auto-fill the registration form when a returning guest arrives." Underlying need: reduce check-in friction for returning guests without re-typing everything they already told the hotel on a previous visit, without forcing the receptionist to leave the PMS they already use, without requiring the receptionist to learn a new tool, and without making the guest stand at the desk longer than they have to. The form-fill is the mechanism; the need is the frictionless recognition of a returning guest.

### 5.2 Buyer's underlying need

Stated want: "faster check-in; modernize the front desk." Underlying need: improve guest experience and reduce staff workload enough to justify the cost of the system, achieve competitive parity with bigger chains that already have this capability, and do all of this without paying for a PMS replacement. The system must be visibly faster (the receptionist notices) and visibly modern (the buyer can show it off).

### 5.3 Data subject's underlying need

The guest (data subject) does not interact with the system directly, but their personal data is the system's input. Their underlying need: their data is collected and stored lawfully, used only for the purpose they gave it (faster future check-ins at this hotel), not shared with third parties without consent, secured against breach, and deletable on request. Under NDPR (Nigeria), GDPR (EU), and CCPA (US), the guest also has the right to know what is stored, the right to correct it, the right to delete it, and the right to data portability.

### 5.4 Maintainer's underlying need

Stated want: "maintainable codebase." Underlying need: a small number of clearly-bounded components with explicit contracts between them, so a single developer can hold the whole system in their head; tests at every boundary so changes are safe; documentation that explains why each piece exists, not just what it does. The most fragile piece (the form-fill) must be the most isolated — when a hotel's PMS vendor pushes an update that breaks the form, the fix should touch only the adapter, not the core.

### 5.5 The need that wins

The receptionist's underlying need — frictionless recognition of a returning guest — is the primary value. Every other need either supports it (the buyer pays for it; the maintainer keeps it running) or constrains it (the data subject's rights limit what we can store; the PMS vendor's permission limits how we read). All four supporting needs are non-negotiable; the primary need is what we are building for.

## 6. CIRCLES — Cut

Choose which need wins, and say what is being given up. The PM agent's cut step is explicit about both — the choice and the cost.

### 6.1 What wins

Receptionist-side frictionless recognition wins. Specifically: live-filtered profile search as the receptionist types, with sub-300ms response; one-click selection of the correct profile; one-click fill of the hotel's existing registration form. Total receptionist round-trip budget under 500ms (per Section 8.5 of the Behavioral Specification).

### 6.2 What we give up

Table 3: Explicitly out-of-scope (what we give up by choosing this cut)

| Out of scope | Why | Cost of giving it up |
|---|---|---|
| AI features | Project plan forbids; single-dev maintenance burden; introduces non-determinism | Lose 'smart' suggestions, natural-language search; gain determinism + lower ops cost |
| Automatic duplicate detection | Project plan forbids; high false-positive rate in hospitality data | Some guests have multiple profiles (one per booking source); acceptable for v1 |

| Out of scope | Why | Cost of giving it up |
|---|---|---|
| Spell correction | Project plan forbids; not needed if search uses substring + prefix match | Receptionist must type accurately; minor friction at most |
| Replacing the PMS | Project plan forbids; out of scope commercially | Hotel keeps their existing PMS, with all its quirks |
| Writing to the hotel's database | Project plan forbids; legal/contractual risk with PMS vendor | Adapter is one-way sync; our DB only |
| Multi-language UI in v1 | Out of scope for v1; English first | Hotels in non-English regions use English UI for now |
| Mobile app for receptionists | Out of scope; receptionist works at a desk | Receptionist uses the web UI on a desktop browser |
| Guest-facing portal | Out of scope; not the customer we serve | Guests never log in; their data is read-only from the PMS |
| Loyalty / points program | Out of scope; that is the PMS's job | Loyalty data, if any, is read from the PMS and displayed, not computed |
| Real-time PMS event subscription | Out of scope for v1; complex to integrate | Sync is periodic (configurable interval), not push-based |

### 6.3 What is deferred (not given up)

The following are explicitly deferred to later phases, not permanently excluded. They are Pending in the 80-area checklist and will be revisited at the end of Phase 2.

- Additional PMS adapters beyond QloApps (Mews, Cloudbeds, Apaleo via their public APIs).
- Windows desktop PMS support via a companion application.
- Card 3's proactive timeline (T+0 WELCOME through T+1d POST_STAY). The phase vocabulary is reserved in our schema; the scheduler is not built in v1.
- Open-sourcing the resulting code (owner's call).

## 7. CIRCLES — List

Produce genuinely different options rather than variations of one. The PM agent's list step is about generating real alternatives — three horizons, each independently shippable.

### 7.1 Three horizons of solutions

Table 4: Solution horizons (each delivers independent value)

| Horizon | What it delivers | Spec cards consumed | Independent value |
|---|---|---|---|
| Quick win (Phase 0) | Foundation: project scaffold, schema (Card 2 + Card 3 phase enum), design system, env config, CI | Card 2, Card 3 | Decisions captured in code; future phases start from a known base |
| Core bet (Phase 1) | Read-side sync adapter (Card 1 contract) + guest profile API (Card 2 six endpoints) + receptionist search screen + admin (sync status, audit log, tenant mgmt) | Card 1, Card 2, Card 4 | Hotel can install the system; receptionist can search and view profiles; data flows from PMS to our DB |
| Core bet (Phase 2) | Browser-extension form-fill adapter (Card 5 algorithm) | Card 5 | Receptionist can one-click fill the hotel's registration form — the full value proposition is now live |
| Longer horizon (Phase 3) | Additional PMS adapters (API-based: Mews, Cloudbeds, Apaleo; Windows desktop via companion app) | Re-applies Card 1, Card 4 patterns | System reaches hotels on other PMS platforms; horizontal scale |

## 7.2 What makes these genuinely different

Each horizon is not a refinement of the previous — it adds a capability that did not exist before. Phase 0 makes no product, only decisions. Phase 1 makes a product that shows profiles but cannot fill forms. Phase 2 adds the fill capability. Phase 3 expands to other PMS systems. If Phase 2 never ships, Phase 1 still has standalone value (faster guest lookup at the desk, even without auto-fill). If Phase 3 never ships, Phase 2 still has full value for one hotel.

## 7.3 Alternatives considered and rejected

The PM agent's list step also names alternatives that were considered and rejected, with reasons. This protects against re-litigating the same options later.

- Build everything at once (no phases). Rejected because single-dev bandwidth; phases let us ship value incrementally and learn between phases.
- Build the form-fill adapter first (Card 5 alone). Rejected because without the profile DB and search screen, the adapter has nothing to fill from. Card 5 depends on Cards 1+2.

- Use a generic ORM-based sync tool (Airbyte, Debezium). Rejected because none matches SSLR's read-only / no-server-config contract; all require either logical replication slots or server-side extensions on the source PMS, which the vendor will not grant.
- Use Cognito for auth (Card 2's stack). Rejected because AWS dependency adds ops cost; JWT-with-group-claims gives the same isolation guarantee with no managed-service dependency.
- Use SQLite for v1 deploy simplicity. Rejected because SSLR's contract (Card 1) is PostgreSQL-native (xmin system column, array_agg, unnest, md5 hash). Re-implementing against SQLite would diverge from the spec card. SQLite is fine for tests; Postgres for production.

## 8. CIRCLES — Evaluate

Weigh the options against what matters, trade-offs included. The PM agent's evaluate step scores the surviving options against the criteria, and is explicit about the trade-offs.

### 8.1 Evaluation criteria

Table 5: Evaluation criteria (derived from constraints + customer needs)

| Criterion | Weight | Why this weight |
|---|---|---|
| Receptionist round-trip latency (p95) | High | Primary value; must be under 500ms end-to-end |
| Single-developer maintainability | High | Hard constraint; cannot be deferred |
| Read-only on source PMS | High | Hard constraint; legal/contractual |
| NDPR / GDPR / CCPA compliance | High | Hard constraint; regulatory |
| Time to first value (Phase 1 ship) | Medium | Earlier value de-risks the project; not at the cost of correctness |
| Cost of v1 infrastructure | Medium | Owner's budget unknown; assume $0–$50/month for v1 |
| Extensibility to other PMS adapters | Medium | Phase 3 depends on this; not blocking v1 |
| Frontend polish (art-directed) | Medium | Buyer's stated want; receptionist's secondary need |
| Time to Phase 2 (form-fill) | Low | Phase 1 alone has standalone value; Phase 2 can ship later |

### 8.2 Trade-offs in the recommended plan

**Table 6: Trade-offs accepted by the recommended plan**

| Trade-off | We chose | We gave up | Why |
|---|---|---|---|
| Postgres vs SQLite | Postgres | SQLite's zero-config deploy | Spec card 1 is Postgres-native; re-implementing against SQLite would diverge from spec |
| JWT vs Cognito | JWT with group claims | Cognito's managed UI + admin tooling | Avoids AWS dependency; group-claim isolation pattern is identical (Card 2 contract) |
| Monorepo vs polyrepo | Monorepo (pnpm workspaces) | Polyrepo's independent versioning | Single dev; shared types; one CI pipeline; one PR to ship a cross-cutting change |
| Next.js API routes vs separate API | Next.js API routes (single deploy) | Separate API service | Single deploy unit; fewer moving parts; sufficient for v1 scale |
| Card 2 envelope vs simpler | Simpler success/failure shape | Card 2's standard envelope consistency | Single dev; less ceremony per handler; can adopt envelope later if integrations need it |
| Browser extension: V3 Chrome-only vs cross-browser | V3 Chrome-only for v1 | Firefox / Safari support in v1 | Manifest V3 is required for Chrome Web Store; Firefox supports V3 with caveats; Safari deferred |
| Adapter-Read + Adapter-Fill: separate vs unified | Separate components | Code reuse between read and fill paths | They share nothing except the guest-data contract; unifying couples unrelated concerns |
| Phase 0 scope: minimal vs comprehensive | Minimal (schema + scaffold + design system) | Comprehensive (includes admin scaffolding) | Phase 0 should ship in days, not weeks; admin scaffolding belongs in Phase 1 |

## 8.3 Risks identified in the evaluation

Table 7: Top risks and mitigations

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Hotel's PMS vendor refuses read-only DB access | Medium | High | Adapter contract supports API-based access as fallback (Phase 3 path); first hotel must confirm access before Phase 1 starts |
| PMS vendor pushes a UI update that breaks the form-fill | High | Medium | Adapter-Fill is isolated (Card 5 algorithm); fix touches only the adapter; core is untouched |
| Receptionist round-trip exceeds 500ms budget | Low | High | Card 2 small-payload pattern + Card 5 sub-100ms solver + Postgres FTS indexes; load-test before Phase 1 ship |
| NDPR/GDPR/CCPA compliance gap discovered late | Low | High | Privacy-by-design baked into schema (consent fields, deleted_at, audit_logs); legal review before any production deploy |
| Single-developer bandwidth | High | Medium | Phased delivery; each phase ships independently; OpenSpec (Step #4) reduces rework |
| Schema drift in source PMS (QloApps upgrade changes columns) | Medium | Medium | Adapter's schema-validation step (Card 1 contract) detects drift; abort or full resync per resyncOnSchemaChange flag |

## 9. CIRCLES — Summarize

Close with one recommendation, one metric, and the reasoning behind it. The PM agent's summarize step is the document's deliverable — everything else supports this.

### 9.1 The recommendation

Build the Guest Profiling System in four phases — Phase 0 (foundation), Phase 1 (read-side adapter + guest API + receptionist search screen + admin), Phase 2 (browser-extension form-fill adapter), Phase 3 (additional PMS adapters). Use Postgres for the target database, Next.js 16 with API routes for the core, TypeScript throughout, Tailwind 4 + shadcn/ui for the frontend, and a Chrome Manifest V3 extension for the form-fill adapter. Build the adapter-read and adapter-fill as separate components with a shared guest-data contract. Adopt Card 2's guests table schema with Card 3's 12-phase enum, plus a source_ref column

for adapter mapping. Build multi-tenant isolation in from day one, even though v1 deploys single-tenant.

### 9.2 The metric

The single metric that defines success: p95 receptionist round-trip latency, measured from the first keystroke in the search box to the registration form being filled. Target: under 500 milliseconds. Composition: search response < 300ms (Card 2 small payload + Postgres FTS) + profile fetch < 100ms (indexed by UUID) + form-fill < 100ms (Card 5 sub-100ms solver). This metric is testable end-to-end and is the acceptance criterion for Phase 2 ship.

### 9.3 The reasoning

Three reasons this recommendation is the right one. First, each phase delivers independent value — if Phase 2 never ships, Phase 1 still reduces check-in friction by making guest lookup faster; if Phase 3 never ships, Phases 1+2 still serve one hotel fully. Second, each phase maps cleanly to one or more of the five reverse-engineered spec cards, which means the behavioral contract is already specified before any code lands — there is no invention step between Plan and Build. Third, the constraints (read-only source, single-dev, NDPR/GDPR/CCPA, no AI, no auto-dedup) are honored throughout: the adapter is one-way, the stack is familiar and well-documented, the schema bakes in consent + soft-delete + audit, the ranking is deterministic, and each source row maps to exactly one profile.

### 9.4 What the recommendation does NOT decide

Per the PM agent's standing rule, the recommendation is not a decision. The following are Pending the owner's call:

- Hosting provider for v1 (Render, Fly.io, Railway, or self-hosted VPS). Recommendation: any Postgres-capable PaaS with managed TLS.
- First hotel customer (must confirm read-only DB access or API access before Phase 1 build starts).
- Whether to open-source the resulting code. Recommendation: MIT license if open-sourced; Apache 2.0 if the owner wants patent protection.
- Brand identity (logo, name beyond "Guest Profiling System"). Recommendation: defer to Phase 1 ship; placeholder for now.

## 10. Blueprint Plan Mode — Constraints (Area 75)

Per the Blueprint skill's Plan mode rules, area 75 (constraint identification) and area 01 (project setup) are read before anything else. Money, hosting, connectivity, and the client's devices decide what is possible.

**Table 8: Constraint inventory (Blueprint area 75)**

| Constraint category | Constraint | Source / rationale |
|---|---|---|
| Money | v1 infrastructure budget: $0–$50/month target | Owner's budget unknown; assume low. Single-tenant Postgres + Next.js fits in this range on Render/Fly.io |
| Money | Development cost: single developer, evenings + weekends | Project plan: single-dev maintainable; no team budget |
| Hosting | PaaS with managed Postgres (Render, Fly.io, Railway) for v1 | Avoids DevOps burden; managed TLS; managed backups; sufficient for single-tenant v1 |
| Connectivity | Source PMS must be reachable from our sync job (network-level) | Either same VPC as source DB, or source DB exposes read-only port. Hotel IT confirms |
| Connectivity | Receptionist browser must reach our API over HTTPS | Standard. No on-prem requirement; cloud-hosted API is fine |
| Client devices | Receptionist workstation: modern Chrome/Edge/Firefox on Windows or Mac | Per project plan; no mobile; no IE; no Safari in v1 |
| Client devices | Hotel IT may impose firewall / proxy rules | Browser-extension install requires IT sign-off; documented in handoff |
| Time | Phase 0: 1–2 weeks. Phase 1: 4–6 weeks. Phase 2: 2–3 weeks. Phase 3: open-ended | Single-dev estimates; not commitments |
| Legal | NDPR (Nigeria), GDPR (EU), CCPA (US) compliance required | Project context; Lagos TZ. Affects data residency, consent, deletion, breach response |
| Legal | Read-only DB access agreement with hotel's PMS vendor | Contractual; adapter cannot ship without it |
| Skill | TypeScript / Node.js / React / Postgres | Owner's stated skill set per project context; not a constraint to negotiate |
| Skill | Browser extension development (Manifest V3) | New skill area; Card 5 spec card provides the algorithm; MIT-licensed reference impl exists |

## 11. Blueprint Plan Mode — Project Setup (Area 01)

Area 01 sits at Project Ring but its hosting/environment answers also feed Engine: Runtime environment and Engine: Deployment. The decisions here shape every downstream area.

Table 9: Project setup decisions (Blueprint area 01)

| Decision | Choice | Rationale |
|---|---|---|
| Monorepo structure | pnpm workspaces; packages: core, adapter-read, adapter-fill-extension, web, shared | Single deploy unit for core+web; adapter-read is a separate worker; extension is bundled separately; shared holds types and contracts |
| Language | TypeScript 5.x throughout | Type safety end-to-end; matches Card 5's stack (TypeScript); familiar to owner |
| Runtime | Node.js 20 LTS (LTS required for production) | Long-term support; matches Next.js 16 requirement |
| Web framework | Next.js 16 (App Router) with API routes | Single deploy unit for frontend + API; sufficient for v1 scale; can split later if needed |
| Database | PostgreSQL 15+ | Spec card 1 contract fidelity; xmin, array_agg, unnest, md5 |
| ORM / query builder | postgres.js (driver) + Kysely (query builder) | Type-safe; minimal abstraction; close to the SQL (needed for SSLR-pattern queries) |
| Migrations | node-pg-migrate | Plain SQL migrations; no ORM lock-in; matches Card 2's migration file pattern |
| Frontend styling | Tailwind CSS 4 + shadcn/ui | Matches user's design brief ("beautiful, polished, modern"); shadcn/ui gives composable primitives |
| Browser extension | Manifest V3; TypeScript; esbuild bundler | Required for Chrome Web Store; matches Card 5's stack |
| Version control | Git; GitHub; trunk-based development | Per project plan; small commits; short-lived branches |
| CI | GitHub Actions; typecheck + lint + test on every PR | Single-dev safety net; catches regressions before merge |
| Testing | Vitest (unit) + Playwright (e2e) + pg-mem (DB unit) | Fast feedback; e2e for the receptionist round-trip; pg-mem for isolated DB tests |

## 12. Blueprint Plan Mode — Decide-First: Multi-Tenancy (Area 49)

One of the three decide-first areas. Per the Blueprint skill, area 49 must be settled before the Data layer because it shapes every table. Even though v1 deploys single-tenant, the isolation boundary is built in from day one — retrofitting it is materially more expensive than building it now.

Decision: every domain table carries a tenant_id column (UUID, FK to tenants table). Every list / collection query goes through a single shared tenant-scope resolver before issuing SQL. The resolver fails closed — if the caller's JWT does not establish a qualifying scope (single tenant, multi-tenant, or admin), the call is rejected. The pattern is exactly Card 2's resolve_property_scope (Section 4.5 of the Behavioral Specification), re-implemented in TypeScript with JWT group claims instead of Cognito.

Table 10: Multi-tenancy decision (Blueprint area 49)

| Question | Answer |
|---|---|
| Multi-tenant in v1 deploy? | No. v1 deploys single-tenant (one hotel per instance). tenant_id is always the same value |
| Multi-tenant in v1 schema? | Yes. Every domain table has tenant_id. Indexes include tenant_id where it filters queries |
| Multi-tenant in v1 query path? | Yes. Every list query calls resolveTenantScope(event) before SQL. Failure mode: 403 Forbidden |
| When does multi-tenant deploy happen? | Phase 3 or later, after Phase 2 ships. Driven by customer demand, not technical readiness |
| How is tenant_id assigned at signup? | Each hotel install creates a tenants row; the install's tenant_id is baked into its env config |
| How does JWT carry tenant scope? | JWT includes tenant_id and groups[] claims; resolver checks groups first, then tenant_id narrows |
| What about super-admin (chain-level) users? | Reserved group claim 'Admin' bypasses tenant filter. Used only for our own ops, not exposed to hotels |

## 13. Blueprint Plan Mode — Decide-First: Delete Policy (Area 59)

Second decide-first area. Per the Blueprint skill, area 59 must be settled before the Data layer because soft-delete vs hard-delete shapes every query and every audit log entry.

Decision: soft delete throughout. Every domain table has a deleted_at timestamp column (nullable; null = active, non-null = soft-deleted). No row is ever hard-deleted by application code. The only hard-delete path is a documented data-retention job, run manually, that purges rows whose deleted_at is older than the retention window (default 7 years per NDPR/GDPR). Source PMS rows that disappear (QloApps soft-deletes a customer by setting

deleted=1) are mirrored on our side by setting deleted_at — never by deleting our row. This preserves the audit trail and honors data-subject portability rights.

Table 11: Delete policy decision (Blueprint area 59)

| Question | Answer |
|---|---|
| Soft or hard delete? | Soft. deleted_at timestamp on every domain table |
| Are soft-deleted rows returned by default? | No. All queries append WHERE deleted_at IS NULL unless explicitly requesting archived rows |
| Source row hard-deleted — what do we do? | Set our deleted_at. Never delete our row. Audit log records the source-side deletion |
| Data subject requests deletion (GDPR/NDPR) | Set deleted_at. Retain for the legal retention window (7 years). Document the request in audit_logs. After window, hard-delete via manual job |
| Audit log retention | Indefinite (no deleted_at on audit_logs). Audit logs are append-only |
| Sync state retention | Indefinite. The replication cursor (Card 1's __sslr_state equivalent) is never deleted |
| Hard-delete path | Manual SQL job, documented in runbook, run by maintainer only. Not exposed via API |

## 14. Blueprint Plan Mode — Build Layers

Per the Blueprint skill's Plan mode, Build Layers are walked in build order (Functionality → Business logic → Data → UI), confirmed by Victor on 2026-11-08. For each layer we answer the questions from references/build-layers.md and the UX Thread questions.

### 14.1 Layer 1 — Functionality (the feature list)

What the user can do. Functionality is the feature list — the Feature Card is the unit that connects all the layers. Each feature below becomes a Feature Card in Section 17.

Table 12: Feature list (Blueprint Functionality layer)

| # | Feature | User | Trigger | Outcome |
|---|---|---|---|---|
| F1 | Receptionist live search | Receptionist | Types in search box | Live-filtered list of matching guest profiles, <300ms response |

| # | Feature | User | Trigger | Outcome |
|---|---|---|---|---|
| F2 | Profile select + view | Receptionist | Clicks a profile in the list | Full guest profile displayed: name, contact, address, preferences, reservations, loyalty, phase |
| F3 | Auto-fill registration form | Receptionist | Clicks "Fill form" button | Hotel's registration form is filled via Card 5 algorithm; high-confidence fields filled silently; medium-confidence highlighted amber for review |
| F4 | Read-side sync job | System | Cron (configurable, default every 5 min) | Adapter pulls changes from source PMS; transforms; writes to our DB. Idempotent; throttled |
| F5 | Tenant management (admin) | Maintainer | Visits admin /tenants | List tenants; create new tenant (for new hotel install); view sync status per tenant |
| F6 | Sync status dashboard (admin) | Maintainer | Visits admin /sync | View last sync per table; view xmin cursor; view throttle utilization; manual trigger sync |
| F7 | Audit log (admin) | Maintainer | Visits admin /audit | View all read/write actions on guest data; filter by tenant, by user, by date range |
| F8 | User management (admin) | Maintainer | Visits admin /users | Create receptionist accounts; assign to tenant; assign groups |
| F9 | Data subject request (admin) | Maintainer | Submits DSR via admin | Export or delete a guest's data per NDPR/GDPR/CCPA. Deletion sets deleted_at; export returns JSON |
| F10 | Schema validation on sync | System | Per sync pass, per table | Detect schema drift in source PMS; abort or full-resync per resyncOnSchemaChange flag |

## 14.2 Layer 2 — Business logic

The rules behind the features. Business logic is what enforces invariants, computes derived values, and makes decisions the user does not directly control.

Table 13: Business logic rules (Blueprint Business logic layer)

| Rule | Feature(s) | Behavior |
|---|---|---|
| Live search ranking | F1 | Order: exact email match > exact phone match > prefix firstname/lastname match > substring firstname/lastname match. No fuzzy matching, no dedup. Limit 20 results. Indexes: trigram (pg_trgm) on email/firstname/lastname, btree on phone |
| Source → profile field mapping | F4 | QloApps id_customer (int) → source_ref (string). firstname + lastname → our name fields. email → email (lowercased). phone → phone (E.164 normalized). birthday → date_of_birth (0000-00-00 → null). is_guest → boolean flag. deleted → deleted_at (set when source.deleted=1) |
| Sync cursor advancement | F4 | Per Card 1: cursor advances only after a complete source transaction is consumed. State row in sync_state table. xmin value persisted after each chunk's transaction commits |
| Tenant scope resolution | All list endpoints | Per Card 2: derive from JWT groups first; tenant_id claim only narrows further. Fail closed if groups do not establish scope. Single shared resolver function |
| Phase derivation | F2 | Per Card 3: derive guest_phase from our reservations table, not pushed by PMS. Rules: future reservation + check_in <= today+2 = PRE_CHECKIN; check_in == today = CHECKIN_DAY; check_in <= today <= check_out = IN_HOUSE; check_out < today, last 7 days = POST_STAY; otherwise IDLE. Computed on read, not stored |
| Form-fill confidence scoring | F3 | Per Card 5: per (form field, profile key) score = exact phrase +35, regex +20, fuzzy similarity × 15 if > 0.6, autocomplete +45, block context +15, negative −40. Source weights: label 1.0, placeholder 0.9, name 0.8, id 0.7, legend 0.6. Take max across sources. Solve with Kuhn-Munkres |
| Sensitive-field stripping | F2 | Per Card 2: identity_doc_type, identity_doc_number, identity_doc_expiry are not returned in GET /guests/{id}. Fetched only via separate privileged flow (admin only) |
| Consent enforcement | F2, F3 | preferences JSONB respected on read; commPrefs.marketing_opt_in must be true before any marketing fields fill into registration form |
| Audit log entry | All mutations | Every write to guest data: who, when, what (before/after), tenant_id. Append-only. Never edited, never deleted |

| Rule | Feature(s) | Behavior |
|---|---|---|
| Data subject request (DSR) handling | F9 | Export: assemble JSON of all guest data (profile, reservations, audit log entries for that guest). Deletion: set deleted_at; record DSR request in audit_logs; schedule manual hard-delete after retention window |

## 14.3 Layer 3 — Data

What is stored. The Data layer is decided by the Functionality + Business logic layers above; the schema serves the rules. Tables below are derived from Card 2's guests table (Section 4.3 of the Behavioral Specification) plus Card 3's phase enum plus our additions (source_ref, tenant_id, deleted_at, audit_logs, sync_state).

Table 14: Data layer — tables and their purpose

| Table | Purpose | Card source | Notes |
|---|---|---|---|
| tenants | One row per hotel install | New (our addition) | tenant_id UUID PK; name; created_at |
| users | Receptionist + admin accounts | New | user_id UUID PK; tenant_id FK; email unique per tenant; password_hash; groups[] (text array) |
| guests | The hub table — one row per guest profile | Card 2 guests table (full) | guest_id UUID PK; tenant_id FK; source_ref (string, the source PMS's id); cognito_sub replaced by user_id linkage pattern; first_name, last_name, email unique per tenant, phone, date_of_birth, preferences JSONB, comm_prefs JSONB, vip_level, total_stays, total_spend, last_stay_date, tags text[], merged_from_ids uuid[], guest_phase (Card 3 enum, computed on read), created_at, updated_at, deleted_at |

| Table | Purpose | Card source | Notes |
|---|---|---|---|
| reservations | Linked spoke — a guest's reservation history | Card 2 pattern (linked) | reservation_id UUID PK; guest_id FK; tenant_id FK; check_in date; check_out date; room_type; status; source_ref (string); created_at, updated_at, deleted_at |
| guest_identity_documents | Linked spoke — identity docs (passport, etc.) | Card 2 pattern | document_id UUID PK; guest_id FK; tenant_id FK; type; document_number (encrypted at rest); expiry_date; created_at, updated_at, deleted_at |
| loyalty_profiles | Linked spoke — loyalty points and tier | Card 2 pattern | loyalty_id UUID PK; guest_id FK; tenant_id FK; tier; points_balance; total_stays; created_at, updated_at |
| audit_logs | Append-only record of every mutation to guest data | Card 2 pattern (implied) | log_id UUID PK; tenant_id FK; actor_user_id FK; action (enum: read, create, update, delete, dsr_export, dsr_delete); entity_type; entity_id; before JSONB; after JSONB; created_at. NO deleted_at column — append-only |
| sync_state | Per-table replication cursor (Card 1's __sslr_state equivalent) | Card 1 | table_name varchar PK (within tenant scope); tenant_id FK; last_seen_xmin bigint; where_clause text; updated_at |
| adapter_connections | Per-tenant source PMS connection config | New | connection_id UUID PK; tenant_id FK; pms_type (enum: qloapps, mews, cloudbeds, custom); connection_url_encrypted; is_read_only boolean (always true); created_at, updated_at |
| schema_migrations | node-pg-migrate's tracking table | Tooling | Auto-managed; not domain data |

Indexes: trigram (pg_trgm extension) on guests(email), guests(first_name), guests(last_name) for live search; btree on guests(phone) for exact match; btree on guests(tenant_id, deleted_at) for the default active-guests-in-tenant query; btree on reservations(guest_id, check_in desc) for the linked-reservations query; btree on audit_logs(tenant_id, created_at) for the audit dashboard; btree on sync_state(tenant_id, table_name) for the per-tenant cursor.

## 14.4 Layer 4 — UI

How it looks. The UI layer is decided last in build order, but the design system is decided now (Phase 0) because every later screen depends on it. The user's design brief was explicit: visually beautiful, polished, modern, professionally art-directed, intentionally crafted, not a generic template. Tailwind 4 + shadcn/ui gives the primitives; the design system on top is our own.

Table 15: UI layer decisions

| Decision | Choice | Rationale |
|---|---|---|
| Design system foundation | Custom token set on top of Tailwind 4; tokens for color, type, spacing, radius, shadow | User brief requires "intentional design decisions"; off-the-shelf Tailwind defaults feel generic |
| Color palette | Single accent (deep teal #1B6B7A from DM-1 palette used in earlier deliverables); neutral grays; semantic colors for success/warning/error | Matches earlier deliverables; professional; not trendy; works for hospitality (calm, trustworthy) |
| Typography | Inter for body; Inter Tight for headings (or similar); monospace for codes/IDs (Courier New for compatibility) | Modern; highly legible at small sizes; available as variable fonts; pairs well |
| Spacing scale | 4px base; consistent 4/8/12/16/24/32/48/64 vertical rhythm | Predictable; matches Tailwind's defaults; avoids ad-hoc spacing |
| Component library | shadcn/ui (Radix primitives + Tailwind) | Composable; accessible (WAI-ARIA); customizable; not a black box; matches Card 5's framework-safe philosophy |
| Layout grid | 12-column on desktop; collapses to single column on tablet; receptionist screen is desktop-only | Receptionist works at a desk; no mobile required for v1 |

| Decision | Choice | Rationale |
|---|---|---|
| Receptionist screen | Two-pane: left = search + results; right = selected profile detail. "Fill form" button at top right of profile pane | Matches F1+F2+F3 flow; no page navigation; minimal clicks |
| Admin screens | Single-page app; sidebar nav: Tenants, Sync, Audit, Users, DSR. Table-heavy; minimal decoration | Maintainer-facing; function over form; consistent with shadcn/ui DataTable |
| Browser extension popup | Small popup (~400px wide): profile picker → "Fill form" button. Confirmation toast on fill | Receptionist's secondary interaction point; primary interaction is the receptionist web screen |
| Empty states + loading | Skeletons (not spinners) for all async content; empty states with helpful illustrations + next-step CTAs | User brief: "refined details, including borders, shadows and subtle interactions"; skeletons are a polish signal |
| Motion | Subtle (150–250ms ease-out); used for transitions, hover states, focus rings. No large animations | User brief: "subtle interactions"; receptionist screen must feel snappy, not theatrical |

## 15. Blueprint Plan Mode — Engine (10 Parts)

Per the Blueprint skill, after the Build Layers we walk the Engine. The Engine is how the product is built, run, and protected. It has 10 parts: Architecture, Integration, Communication between parts, Runtime environment, Deployment, Configuration, Security, Reliability, Scalability, Resource management.

### 15.1 Engine: Architecture

Monorepo with five packages: core (Next.js app + API), adapter-read (sync worker, separate process), adapter-fill-extension (Chrome MV3 extension, separate build), web (the receptionist UI, lives inside core as Next.js pages), shared (types + contracts + Zod schemas). The adapter-read runs as a separate process because it has its own connection pool to the source PMS and its own throttle loop; mixing it into the Next.js request path would couple request latency to sync load.

### 15.2 Engine: Integration

Three integration surfaces. First, the adapter-read connects to the source PMS database via postgres.js with the connection URL stored encrypted in adapter_connections. The connection MUST use default_transaction_read_only=true (per Card 1's contract). Second,

the browser-extension adapter-fill connects to our core API over HTTPS; it authenticates with a per-extension JWT issued at install time. Third, the receptionist web UI is the same Next.js app that serves the API — no separate frontend-to-backend integration.

## 15.3 Engine: Communication between parts

Table 16: Communication patterns (Engine gap questions C1–C3)

| Communication | C1: How do they talk? | C2: What if late or lost? | C3: What if duplicated? |
|---|---|---|---|
| Receptionist web → API | HTTPS REST, JSON | Request times out (5s); UI shows error toast; retry on user action | Idempotent reads; duplicate GET returns same data |
| Extension → API | HTTPS REST, JSON, Bearer JWT | Same as above | Same as above |
| Adapter-read → source PMS | Postgres connection (postgres.js) | Connection drops mid-pass; SSLR-pattern resumes from last xmin cursor; safe | Idempotent upsert per chunk (DELETE matching PKs + CopyFrom). Re-running same chunk is safe |
| Adapter-read → our DB | Postgres connection (postgres.js) | Transaction rollback; chunk retried on next sync pass | Same idempotent upsert; safe |
| Cron → adapter-read | Cron triggers the worker process | If cron misses a tick, next tick picks up from cursor | If two workers run for same tenant, last-writer-wins on cursor; documented constraint: one worker per tenant |

## 15.4 Engine: Runtime environment

Where it runs. Node.js 20 LTS for core + adapter-read; Chrome MV3 for the extension; PostgreSQL 15+ for our DB. Three environments: dev (local Docker Compose), staging (PaaS preview), production (PaaS). Limits: API request timeout 30s; sync job wall-clock 10 min max per tenant per pass; adapter-read memory ceiling 512MB. Real conditions: hotel networks may be slow or unreliable; receptionist workstation may be old; the sync job must tolerate interruptions gracefully (Card 1 contract: state row persists last-seen xmin).

## 15.5 Engine: Deployment

PaaS deployment (Render or Fly.io — owner's call, Pending). Single deploy for core + adapter-read worker (PaaS supports worker processes alongside web services). Database is managed Postgres on the same PaaS. TLS terminated at the PaaS edge. Database backups:

daily automated, 7-day retention default, point-in-time recovery enabled if the PaaS supports it. Extension is published to Chrome Web Store (separate build, manual upload for v1; automated via GitHub Actions later).

## 15.6 Engine: Configuration

Environment variables per environment (dev / staging / production). Secrets (DB URLs, JWT signing keys, source PMS credentials) stored in PaaS-managed env vars, never in the repo. Per-tenant config (adapter connection URLs) stored in the adapter_connections table, encrypted at rest with a key from env. No .env files in production; .env.example in the repo for documentation only.

## 15.7 Engine: Security

JWT with group claims for auth (per Card 2 pattern, simplified from Cognito). Passwords hashed with Argon2id (memory-hard, resistant to GPU attacks). TLS 1.3 enforced for all HTTP traffic. Database connection requires SSL. Source PMS connection uses default_transaction_read_only=true (Card 1 contract). Read-only database user on source PMS, configured by hotel IT. Rate limiting on all API endpoints (60 requests per minute per JWT). No CORS for v1 (extension and web share origin). Audit log entries on every mutation. Sensitive fields (identity documents) encrypted at rest with application-level encryption (pgcrypto or application-layer AES-GCM).

## 15.8 Engine: Reliability

Per Card 1: sync job is idempotent (DELETE + CopyFrom in a single transaction per chunk). Per Card 1: state row persists last-seen xmin; resume after crash. Daily backups, 7-day retention. Point-in-time recovery if PaaS supports it. Health check endpoint GET /health returns 200 if DB is reachable and last sync was within 2x the configured interval. Alert if health check fails for 5 minutes (PaaS-built-in or simple uptime monitor).

## 15.9 Engine: Scalability

v1 single-tenant: one hotel, ~10,000 guests, ~100,000 reservations over 5 years. Well within Postgres single-instance capacity. Designed for horizontal scale at Phase 3: tenant_id column on every table enables per-tenant partitioning later. Read replicas for the receptionist search screen if a single instance cannot keep p95 under 300ms at 100 hotels — not needed in v1. No premature optimization.

## 15.10 Engine: Resource management

Connection pooling: postgres.js with max 10 connections per pool, one pool per database (our DB + source PMS). Adapter-read throttle: per Card 1's 2× overshoot-sleep formula, default 75% source utilization. Receptionist API: simple in-memory LRU cache (max 1000 entries, 60s TTL) for GET /guests/{id} — invalidate on update. Browser extension memory budget: <50MB. Next.js bundle: target <300KB initial JS for the receptionist screen.

## 16. Blueprint Plan Mode — Project Ring (12 Areas)

The Project Ring is the work around the product: client, version control, legal, docs, handoff, progress tracking. It sits around everything. Per the Blueprint skill, the Project Ring has 12 areas in the 80-area checklist — the table below covers all of them.

**Table 17: Project Ring — 12 areas decided**

| # | Area | Decision |
|---|---|---|
| 01 | Project setup | Per Section 11. pnpm monorepo, TypeScript, Node.js 20, Next.js 16, Postgres 15+, Tailwind 4, shadcn/ui, Vitest + Playwright |
| 15 | Version control conventions | Git; GitHub; trunk-based; conventional commits (feat/fix/docs/chore/refactor/test); squash-merge PRs; main is always deployable |
| 16 | Development workflow | Feature branches off main; PR + CI required; self-merge allowed if CI green; no direct commits to main |
| 17 | Testing | Vitest unit (functions, business logic); pg-mem for DB unit; Playwright e2e for receptionist round-trip + form-fill; 80% line coverage target on shared/ and core/src/server/ |
| 20 | Legal and compliance | NDPR (Nigeria), GDPR (EU), CCPA (US) compliance required. Privacy policy + DSR process documented before any production deploy. Read-only DB access agreement with hotel's PMS vendor (contractual) |
| 21 | Documentation | README + ARCHITECTURE.md (this plan lives there); OpenAPI 3.1 spec for the API; runbook for sync job, DSR, schema drift; ADRs for major decisions |
| 22 | Naming conventions | TypeScript: camelCase for vars/functions, PascalCase for types/components, SCREAMING_SNAKE for constants. DB: snake_case for tables and columns. API: kebab-case for paths, camelCase for JSON keys. Files: kebab-case for paths, PascalCase for React components |
| 32 | Client communication | Owner communicates via IM (this conversation). Status updates at the end of each Step. Decisions captured in this document and in commit messages |

| # | Area | Decision |
|---|---|---|
| 33 | Handoff and maintenance | Runbook: how to deploy, how to run a sync job manually, how to handle a DSR, how to handle schema drift, how to add a new tenant. Maintenance burden tracked via TODOs in repo + GitHub Issues |
| 73 | Progress tracking | PROGRESS.md file in repo root; updated at end of each session; records what's Decided, what's Pending, what's blocked |
| 74 | Session recovery | PROGRESS.md is the resume point. After any session restart, read PROGRESS.md first; resume from the last completed task. Closes the dead-session gap |
| 75 | Constraint identification | Per Section 10. Money, hosting, connectivity, devices, time, legal, skill — all captured |

## 17. Feature Cards

Per the Blueprint skill, a Feature Card traces one feature through every Build Layer and the Engine. Each card below corresponds to a feature in Section 14.1 (Functionality layer). Cards are concise — they are references, not specifications. The OpenSpec design (Step #4) will produce the detailed spec for each card.

### 17.1 F1 — Receptionist live search

Functionality: receptionist types in a search box; system returns matching guest profiles live as they type, under 300ms response. Business logic: ranking by exact email > exact phone > prefix name match > substring name match; limit 20; no dedup. Data: reads from guests table with trigram indexes (pg_trgm) on email, first_name, last_name; btree on phone. UI: search input with debounce (200ms); results list with avatar circle (initials), name, email, last-stay date; click selects. Engine: GET /guests/search?q=...; tenant-scoped via shared resolver; in-memory LRU cache, 60s TTL, 1000 entries. Card source: Card 2 (GET /guests paginated endpoint adapted for live search). Acceptance: p95 < 300ms on 10k guests in single tenant.

### 17.2 F2 — Profile select and view

Functionality: receptionist clicks a profile; system shows full guest detail. Business logic: SENSITIVE_FIELDS stripped from GET /guests/{id} response (per Card 2); guest_phase computed on read (per Card 3 derivation rules). Data: hub-and-spoke — main query hits guests; separate queries for /reservations, /loyalty, /identity-documents (admin only). UI: right pane of receptionist screen; sections for Contact, Address, Preferences, Reservations (last 5), Loyalty, Phase badge. Engine: GET /guests/{id}, GET /guests/{id}/reservations,

GET /guests/{id}/loyalty; owner-only (caller.sub == guest.user_id) or staff with tenant scope. Card source: Card 2.

### 17.3 F3 — Auto-fill registration form

Functionality: receptionist clicks "Fill form"; hotel's existing registration form is filled via Card 5's three-stage pipeline. Business logic: scoring formula per Card 5 (exact +35, regex +20, fuzzy × 15 if > 0.6, autocomplete +45, block context +15, negative −40); Kuhn-Munkres solver on cost matrix (max_score − score); native prototype setter bypass for React/Vue/Angular. Data: reads from guests + guest_identity_documents (if consent allows). UI: Chrome MV3 extension popup; profile picker; "Fill form" button; confirmation toast; high-confidence matches fill silently, medium-confidence highlighted amber for review. Engine: extension authenticates with per-extension JWT; calls GET /guests/{id} then executes Card 5 algorithm locally; sub-100ms solver target. Card source: Card 5.

### 17.4 F4 — Read-side sync job

Functionality: cron-triggered job pulls changes from source PMS into our DB. Business logic: per Card 1 contract — xmin cursor tracking, divide-and-conquer delete detection via md5 PK hashing, 2× throttle formula, idempotent upsert (DELETE + CopyFrom per chunk). Data: writes to guests, reservations, sync_state; source mapping per QloApps (Card 4). UI: none (system feature); admin visibility via F6. Engine: separate worker process; cron every 5 min default; throttled to 75% source utilization; idempotent on retry. Card source: Card 1 + Card 4. Acceptance: single-tenant, 10k guests, full sync < 10 min; incremental sync < 30s.

### 17.5 F5–F10 — Admin features

F5 Tenant management (list, create, view sync status per tenant); F6 Sync status dashboard (per-table cursor, throttle utilization, manual trigger); F7 Audit log (filterable by tenant, user, action, date range); F8 User management (create receptionist accounts, assign tenant, assign groups); F9 Data subject request (export JSON or schedule soft-delete); F10 Schema validation on sync (detect source drift, abort or full-resync per flag). All admin-only, all behind the Admin group claim, all use shadcn/ui DataTable primitives, all tenant-scoped. Card sources: Card 1 (sync state), Card 2 (audit pattern), Card 4 (QloApps schema).

## 18. Blueprint Plan Mode — 80-Area Checklist Status

Per the Blueprint skill's Plan mode rule: "In Plan mode every area ends as Decided, Skip or Pending. Nothing may stay unmarked." The table below marks every one of the 80 areas

plus the 17 gap questions. Sections applied: A (always) + C (web app) + E (always). Sections skipped: B (the project does not sell products) and D (no mobile or desktop app in v1).

Table 18: 80-area checklist — status after Plan mode

| # | Area | Section | Status | Note |
|---|---|---|---|---|
| 01 | Project setup | A | Decided | Per Section 11 |
| 02 | Stack decisions | A | Decided | TS / Node 20 / Next.js 16 / Postgres 15 / Tailwind 4 / shadcn/ui |
| 03 | Design system | A | Decided | Per Section 14.4 — custom tokens on Tailwind 4 |
| 04 | Brand and identity | A | Pending | Owner's call — logo, name beyond "Guest Profiling System" |
| 05 | Frontend architecture | A | Decided | Next.js 16 App Router; React Server Components where possible |
| 06 | Routing and navigation | A | Decided | Next.js file-based routing; /search, /profile/[id], /admin/* |
| 07 | SEO and discoverability | A | Skip | Internal tool, not indexed. robots.txt denies all |
| 08 | Performance | A | Decided | p95 < 500ms receptionist round-trip; bundle < 300KB initial JS |
| 09 | Responsive and cross-device | A | Decided | Desktop-first; tablet graceful degradation; no mobile in v1 |
| 10 | Cross-browser compatibility | A | Decided | Chrome + Edge + Firefox for web; Chrome only for extension (MV3) |
| 11 | Accessibility | A | Decided | WAI-ARIA via Radix primitives; keyboard navigation; focus rings; screen reader tested |
| 12 | Assets and media | A | Decided | SVG icons (Lucide); no images in v1; small bundle |
| 13 | Error handling | A | Decided | Standard error envelope on API; toast on UI; sync job logs errors and continues |
| 14 | Environment and configuration | A | Decided | PaaS env vars; .env.example in repo; no secrets in repo |
| 15 | Version control conventions | A | Decided | Per Table 17 — trunk-based, conventional commits |
| 16 | Development workflow | A | Decided | Per Table 17 — feature branches, PR + CI required |

| # | Area | Section | Status | Note |
|---|---|---|---|---|
| 17 | Testing | A | Decided | Per Table 17 — Vitest + Playwright + pg-mem |
| 18 | Deployment and DevOps | A | Decided | PaaS deploy on merge to main; one-click rollback |
| 19 | Monitoring and observability | A | Decided | GET /health endpoint; PaaS logs; alert if health fails 5 min |
| 20 | Legal and compliance | A | Decided | Per Table 17 — NDPR + GDPR + CCPA; PMS vendor read-only agreement |
| 21 | Documentation | A | Decided | Per Table 17 — README + ARCHITECTURE.md + OpenAPI + runbook + ADRs |
| 22 | Naming conventions | A | Decided | Per Table 17 |
| 23 | Animation and interactions | A | Decided | Subtle 150–250ms ease-out; per Section 14.4 |
| 24 | Notifications and feedback | A | Decided | Toast notifications (sonner); in-app banner for sync failures |
| 25 | Forms and validation | A | Decided | react-hook-form + Zod schemas (shared with API) |
| 26 | Internationalization | A | Skip | English only in v1. Revisit at Phase 3 |
| 27 | Offline and PWA | A | Skip | Receptionist workstation is always online. Not v1 |
| 28 | Cookie and session management | A | Decided | JWT in httpOnly cookie; sameSite=strict; 24h sliding expiration |
| 29 | Redirect and URL management | A | Decided | Next.js middleware for /admin/* auth check; redirect to /login |
| 30 | Code splitting and bundle size | A | Decided | Next.js dynamic imports for admin routes; receptionist route is the critical path |
| 31 | Backup and recovery | A | Decided | Daily automated DB backup; 7-day retention; PITR if PaaS supports |
| 32 | Client communication | A | Decided | Per Table 17 — IM with owner; status at end of each Step |
| 33 | Handoff and maintenance | A | Decided | Per Table 17 — runbook + ADRs + GitHub Issues |

| # | Area | Section | Status | Note |
|---|---|---|---|---|
| 34–42 | E-commerce features | B | Skip | Section B does not apply — project does not sell products |
| 43 | Authentication and authorization | C | Decided | JWT with group claims; Argon2id passwords; per Section 15.7 |
| 44 | User management | C | Decided | Per F8 (Section 14.1) — admin creates receptionist accounts |
| 45 | Onboarding flow | C | Decided | First-run: maintainer sets up tenant + adapter connection; receptionists get an invite link |
| 46 | Dashboard and data visualization | C | Decided | Admin dashboard with sync status (F6), audit log (F7), tenant list (F5) |
| 47 | Real-time features | C | Skip | No real-time needs in v1; periodic sync (every 5 min) is sufficient |
| 48 | Notification system | C | Decided | Toast on UI; in-app banner for sync failures. No email/push in v1 |
| 49 | Multi-tenancy | C | Decided | Per Section 12 — built in from day one; single-tenant deploy in v1 |
| 50 | File management | C | Skip | No file uploads in v1. Identity docs are read-only from PMS |
| 51 | Search and filtering | C | Decided | Per F1 — live search on guests; pg_trgm indexes; limit 20 |
| 52 | Audit logs and activity history | C | Decided | audit_logs table; append-only; per Table 14 |
| 53 | Import and export | C | Decided | Export: DSR JSON (F9). Import: not in v1 (data comes via sync) |
| 54 | Subscription and feature gating | C | Skip | No subscription tiers in v1; all features available to all tenants |
| 55 | Settings and configuration | C | Decided | Per-tenant adapter config in adapter_connections; admin UI to edit |
| 56 | Admin panel | C | Decided | Per F5–F10 — full admin under /admin/* |
| 57 | API design | C | Decided | REST; JSON; OpenAPI 3.1 spec; standard error shape; per Section 15.1 |

| # | Area | Section | Status | Note |
|---|---|---|---|---|
| 58 | Background jobs and queues | C | Decided | Cron-triggered adapter-read worker; no queue in v1 |
| 59 | Soft delete vs hard delete | C | Decided | Per Section 13 — soft delete throughout; manual hard-delete via documented job |
| 60 | Collaboration features | C | Skip | No multi-user collaboration in v1. One receptionist at a time per profile |
| 61 | Webhooks (outgoing) | C | Skip | No outgoing webhooks in v1. Revisit if hotel's PMS supports incoming webhooks |
| 62 | Timezone and localization | C | Decided | All timestamps stored UTC TIMESTAMPTZ; rendered in tenant's configured TZ |
| 63 | Rate limiting and abuse prevention | C | Decided | 60 req/min per JWT; PaaS-level DDoS protection |
| 64 | Multi-step flows and wizards | C | Decided | First-run setup wizard for new tenant; receptionist screen is single-page |
| 65–72 | Mobile / desktop app features | D | Skip | Section D does not apply — no mobile or desktop app in v1 |
| 73 | Progress tracking | E | Decided | Per Table 17 — PROGRESS.md in repo root |
| 74 | Session recovery | E | Decided | Per Table 17 — PROGRESS.md is resume point |
| 75 | Constraint identification | E | Decided | Per Section 10 |
| 76 | Scalability planning | E | Decided | Per Section 15.9 — single-tenant v1; tenant_id enables per-tenant partitioning later |
| 77 | Security | E | Decided | Per Section 15.7 |
| 78 | Landing page vs app shell | E | Decided | App shell — no public landing page in v1. /login redirects to /search after auth |
| 79 | Design tokens and theming | E | Decided | Per Section 14.4 — custom token set on Tailwind 4; single accent (deep teal) |
| 80 | Third party integrations | E | Decided | Source PMS (Card 1); browser extension fill (Card 5). No other integrations in v1 |

Summary of the 80-area checklist: 60 areas Decided, 16 areas Skipped (with reasons), 4 areas Pending (owner's call). No area is left unmarked. Per the Blueprint skill's Plan mode rule, this satisfies the "nothing may stay unmarked" requirement.

### 18.1 Pending areas — questions for the owner

Four areas are Pending the owner's call. Each is a single decision; none blocks Phase 0 work.

- Area 04 (Brand and identity): pick a name beyond "Guest Profiling System" and a logo. Suggestion: defer to Phase 1 ship; use placeholder for now.
- Hosting provider (Render, Fly.io, Railway, self-hosted VPS). Recommendation: any Postgres-capable PaaS with managed TLS and managed backups.
- First hotel customer (must confirm read-only DB or API access before Phase 1 build starts). Recommendation: identify before Phase 1 starts; cannot ship Phase 1 without a real source PMS to test against.
- Open-source license decision (MIT or Apache 2.0 if open-sourced; proprietary if not). Recommendation: MIT if open-sourced; defer decision to after Phase 1 ships.

## 19. Phased Build Plan

The phased build plan is the deliverable from this Step #3 — it becomes the input to Step #4 (OpenSpec design) and the actual code build. Each phase is independently shippable and delivers user-visible value.

### 19.1 Phase 0 — Foundation (1–2 weeks)

Goal: capture decisions in code. No user-visible product yet. By the end of Phase 0, the repo exists, the schema is migrated, the design system is in place, and the CI pipeline passes.

- Monorepo scaffold: pnpm workspaces; packages core, adapter-read, adapter-fill-extension, web (inside core), shared
- Database schema migration: all tables in Table 14; pg_trgm extension; all indexes; trigger for updated_at
- Design system: Tailwind 4 config; shadcn/ui installed; custom token set (color, type, spacing); base components (Button, Input, Card, DataTable, Toast, Badge)
- Auth scaffolding: JWT issue + verify; group-claim resolver; /login + /logout + /register; middleware for /admin/*
- CI pipeline: GitHub Actions; typecheck + lint + unit tests; Playwright smoke test against /login

- Documentation skeleton: README, ARCHITECTURE.md (this document), OpenAPI 3.1 stub, runbook stub, PROGRESS.md
- Acceptance: pnpm install + pnpm build + pnpm test all pass on a fresh clone. Database migration runs cleanly against a fresh Postgres.

## 19.2 Phase 1 — Read-side sync + guest API + receptionist search + admin (4–6 weeks)

Goal: ship a usable product. By the end of Phase 1, a hotel can install the system, the sync job runs, the receptionist can search and view profiles, and the maintainer has admin tools. The receptionist cannot yet auto-fill the form (that is Phase 2), but they can already find a returning guest faster than the PMS's native search.

- Adapter-read (Card 1 contract): SSLR-pattern sync; xmin cursor; divide-and-conquer deletes; throttle; idempotent upserts. Tests against a seeded QloApps DB (Card 4 schema).
- Source PMS mapping (Card 4): QloApps PREFIX_customer → our guests table; field-by-field transformation; soft-delete mirroring; phone normalization to E.164.
- Guest profile API (Card 2 six endpoints, simplified envelope): POST/GET/PUT /guests, GET /guests/{id}/reservations, GET /guests/{id}/loyalty, GET /guests (paginated, PMS-only).
- Tenant scope resolver: Card 2's resolve_property_scope pattern, TypeScript re-implementation, fail-closed.
- Live search endpoint: GET /guests/search?q=…; trigram indexes; ranking per Section 14.2; limit 20; in-memory LRU cache.
- Receptionist web UI: /search screen, two-pane layout, F1+F2 features. Polished, art-directed, per Section 14.4.
- Admin UI: /admin/tenants (F5), /admin/sync (F6), /admin/audit (F7), /admin/users (F8), /admin/dsr (F9). All behind Admin group claim.
- Audit log: append-only on every mutation; tenant-scoped; filterable in admin UI.
- Sync status dashboard: per-table cursor, last sync time, throttle utilization, manual trigger button.
- Schema drift detection (F10): per Card 1's resyncOnSchemaChange flag.
- Acceptance: p95 search < 300ms on 10k guests; sync job completes full pass in < 10 min; receptionist round-trip (without form-fill) < 400ms; admin UI fully functional.

## 19.3 Phase 2 — Browser extension form-fill (2–3 weeks)

Goal: full value proposition. By the end of Phase 2, the receptionist can one-click fill the hotel's existing registration form. This is the moment the system pays for itself.

- Manifest V3 extension scaffold: TypeScript, esbuild, content script + popup + background service worker.
- Scanner + normalizer (Card 5 stage 1): DOM walker; detects input/select/textarea/combobox; extracts 5 signals (label, placeholder, name, id, legend); tokenizer + abbreviations dictionary.
- Signal scorer (Card 5 stage 2): exact +35, regex +20, fuzzy × 15, autocomplete +45, block context +15, negative −40; source weights 1.0/0.9/0.8/0.7/0.6; max across sources.
- Kuhn-Munkres solver (Card 5 stage 2): $O(dim^3)$; cost matrix max_score − score; uniform sentinel for non-candidates; de-duplication of output.
- Native prototype setter (Card 5 stage 3): bypass React/Vue/Angular; dispatch input + change events; pristine-iframe fallback.
- Hospitality-specific rules.json: rules for the hotel registration form fields (first name, last name, email, phone, address, date of birth, nationality, ID type, ID number, etc.).
- Receptionist confirmation flow: high-confidence fields fill silently (green); medium-confidence highlighted amber for click-to-confirm; below LOW_THRESHOLD not filled.
- Per-extension JWT: issued at install time; stored in chrome.storage.local; sent as Bearer token on API calls.
- Acceptance: p95 form-fill < 100ms on a typical hotel registration form (50 fields); receptionist round-trip (full, with form-fill) < 500ms.

## 19.4 Phase 3 — Additional PMS adapters (open-ended)

Goal: reach hotels on other PMS platforms. Each adapter is a separate workstream; the schedule is driven by customer demand, not technical readiness.

- API-based PMS (Mews, Cloudbeds, Apaleo): adapter-read reuses Card 1 contract but the source is an HTTP API, not a Postgres connection. Different change-detection mechanism (API cursor / watermark, not xmin).

- Windows desktop PMS: companion application (Electron or Tauri) that runs on the hotel's workstation; reads the PMS's local database; syncs to our cloud. Form-fill is via UI automation on the desktop PMS.
- Each new adapter: separate package in the monorepo; shares the adapter-read contract; ships independently.
- Acceptance per adapter: full sync < 10 min; incremental < 30s; no false positives in field mapping.

## 20. Hand-off to Step #4 (OpenSpec Design)

This document is the input to Step #4. The OpenSpec architect agent will use the openspec skill family (explore → propose → update-change → sync-specs → archive-change) to formalize the Phase 0 and Phase 1 build as a set of OpenSpec change proposals. Each change proposal will produce spec deltas against the project's main specs, and the build will execute against those specs.

Recommended OpenSpec change proposals for Step #4:

- Change #1: Foundation (Phase 0). Spec deltas for: project scaffold, database schema, design system, auth scaffolding, CI pipeline.
- Change #2: Adapter-read (Phase 1 part 1). Spec deltas for: SSLR-pattern sync worker, QloApps source mapping, sync state table, schema drift detection.
- Change #3: Guest profile API (Phase 1 part 2). Spec deltas for: six guest endpoints, tenant scope resolver, standard error shape, audit log integration.
- Change #4: Receptionist web UI (Phase 1 part 3). Spec deltas for: /search screen, /profile/[id] screen, design system application, accessibility.
- Change #5: Admin UI (Phase 1 part 4). Spec deltas for: /admin/tenants, /admin/sync, /admin/audit, /admin/users, /admin/dsr.
- Change #6: Browser extension form-fill (Phase 2). Spec deltas for: MV3 extension, scanner, scorer, Kuhn-Munkres solver, native setter, hospitality rules.json.

Each change is independent; OpenSpec's archive-change step closes them out one at a time as they ship. The build runs against the specs, not against this document — this document is the recommendation; the OpenSpec deltas are the binding contract for the code.

Final note from the PM agent: this document recommends, it does not decide. Where the owner disagrees with any Decided area, the area becomes Pending with the owner's reason recorded, and Step #4 reflects the revised decision. Where the owner accepts the recommendation as-is, Step #4 may proceed.